

9일 차

전처리

↓
RNN

자연어 처리 기초 RNN

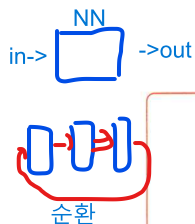
↓
NLP
↓

전처리 → Transformer 사용

이번 차시에서 주목할 키워드

- NLP
- 토큰화 -> 분리(문장, 단어)
- 임베딩 -> 단어를 숫자(벡터)로 변경
- RNN -> 순환 신경망
- 은닉 상태

RNN
↳ LSTM => ...



Text -> 신경망
숫자로
변경
(전처리)

9일 차 학습 목표

- 자연어 처리의 기본 개념과 토큰화, 임베딩 등의 전처리 흐름을 설명할 수 있습니다.
- RNN의 구조와 동작 원리를 이해하고 문장 순서와 문맥 정보 처리를 설명할 수 있습니다.
- 트위터 감정 분류 예제로 자연어 처리 프로젝트의 절차를 직접 설계하고 구현합니다.

text 긍정/부정 => 2진 분류

사람은 문장을 읽고 쉽게 의미를 이해하지만, 컴퓨터는 텍스트를 단순한 문자로만 인식합니다. 예컨대 컴퓨터는 “오늘 기분이 좋아요”라는 문장에서 긍정적인 감정을 파악하려면 복잡한 처리가 필요합니다. 이런 처리 과정을 자연어 처리(Natural Language Processing, 이하 NLP)라고 합니다. 딥러닝에서는 기본적으로 순환 신경망(Recurrent Neural Network, 이하 RNN)을 활용해 문장의 순서를 기억하고 단어 간의 관계를 파악해 감정 분석, 번역, 챗봇 등에 적용합니다.

이번 차시에서는 자연어 처리의 기본 개념을 이해하고 RNN을 활용한 언어 모델링 과정을 실습하면서 NLP 프로젝트의 전체 흐름을 경험합니다. 다만 하나의 차시에서 다루기에 자연어 처리는 너무 방대하므로 개념 중심으로 설명하되 실습은 마지막 절에서 다룹니다. 이번 차시를 마치면 RNN의 기본 원리와 구조, 동작 방식을 이해하고 자연어의 전처리부터 모델링까지의 전체 흐름을 스스로 따라갈 수 있게 됩니다.

자연어 처리 개요

좁은 의미: 텍스트를 컴퓨터가 계산할 수 있는 숫자 형태로 변환하는 과정.

넓은 의미: 변환되는 숫자로 감성을 분석하거나 번역, 요약하는 일 전체

자연어 처리란?

사람은 말하거나 글을 쓸 때 자연스럽게 언어를 사용합니다. 하지만 컴퓨터는 사람의 언어를 바로 이해하지 못합니다. 텍스트는 컴퓨터에게 그저 기호와 문자의 나열일 뿐입니다. 컴퓨터가 인간의 언어를 이해하고 생성하고 대화할 수 있도록 도와주는 기술이 바로 자연어 처리(Natural Language Processing)입니다.

자연어 처리는 좁은 의미로는 텍스트를 컴퓨터가 이해할 수 있는 숫자 형태로 변환하는 과정입니다. 넓은 의미에서는 인간의 언어를 이해하고 처리하는 인공지능의 한 분야로 볼 수 있습니다. 예를 들어 “나는 오늘 기분이 좋아요”라는 문장을 숫자 배열로 바꾸고 이 숫자들을 가지고 감정을 분석하거나 번역하거나 요약하는 일이 바로 자연어 처리입니다.

모호성: "배" 단어만 보고 의미 추론 어렵. 문장 전체를 봐야..
 지시대상 파악: "그녀는 매우 기뻐다." -> "그녀" 만으로 누군지 알 수 X
 언어의 불규칙성: 복잡한 문법 규칙과 예외가 많다. -> 한국어는 어순 바뀌어도 되지만 언어모델은 안 됨.)
 긴 문맥 기억 필요

자연어 처리의 활용과 도전 과제

자연어 처리는 다양한 분야에서 활발히 활용되고 있습니다. 우리가 자주 사용하는 기계 번역 서비스(Google 번역기, Papago), 음성 비서와 챗봇(ChatGPT, Siri), SNS의 감정 분석, 검색 엔진과 추천 시스템, 그리고 AI 뉴스 요약 생성 등이 모두 자연어 처리의 한 분야입니다.

하지만 자연어 처리는 생각보다 쉽지 않은 도전 과제입니다. 같은 단어도 문맥에 따라 전혀 다른 뜻을 가지는 모호성(Ambiguity), 복잡한 문법 규칙과 예외가 많은 언어의 특성, 특정 언어의 데이터 부족 또는 편향성, 그리고 문맥 이해를 위해서는 긴 문장을 파악하며 기억해야 하는 어려움이 자연어 처리에는 존재합니다.

예를 들어 '배'라는 단어는 "배가 아프다"라고 할 때는 신체를 의미하고, "배를 타고 떠나다"라고 할 때는 교통수단을 의미합니다. 또 "그녀는 매우 기뻐다"라는 문장에서 '그녀'가 누구를 지칭하는지 컴퓨터는 혼자서 알아내기 어렵습니다. 이 문제들을 해결하기 위해 자연어 처리 기술은 꾸준히 발전해 왔습니다. 초기에는 문법 규칙이나 확률 통계를 기반으로 한 전통적인 방법이 사용되었지만, 최근에는 딥러닝 기반의 NLP, 특히 RNN, LSTM, Transformer 같은 신경망 모델을 사용해 더욱 정교하고 똑똑하게 발전하고 있습니다. 또한 BERT, GPT처럼 사전 학습된 대규모 언어 모델(Large Language Model, LLM)의 등장으로 컴퓨터는 점점 사람처럼 문장을 읽고 이해하는 능력을 갖추게 되었습니다.

자연어 처리는 사람의 언어를 컴퓨터가 '이해'하도록 만드는 흥미로운 기술 영역입니다. 지금부터 자연어 처리의 기초 개념에서 시작해 텍스트를 숫자로 바꾸는 방법(인코딩과 임베딩), 그리고 텍스트를 이해하고 처리하는 RNN 모델까지 차근차근 알아보겠습니다.

용어 정리

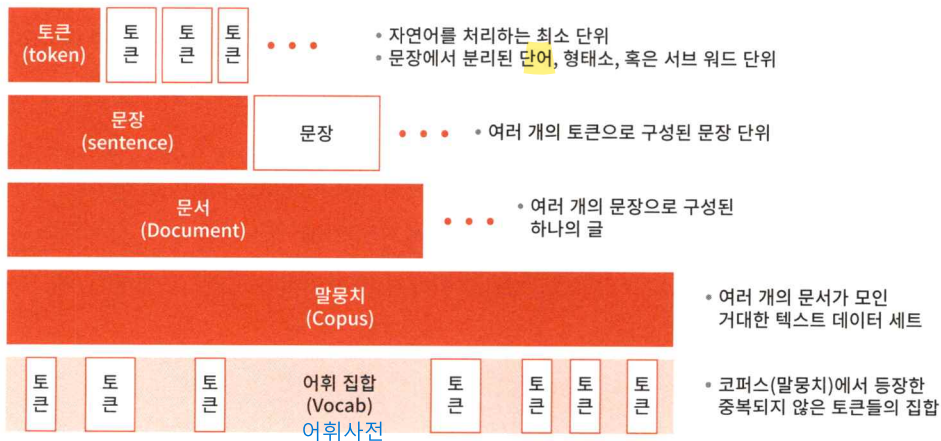
자연어 처리를 시작할 때 가장 먼저 해야 할 일은 텍스트를 컴퓨터가 이해할 수 있도록 변환하는 작업입니다. 문장을 이해하는 일은 사람에게겐 자연스럽지만, 컴퓨터는 그 문장을 '숫자'로 바꿔야 이해할 수 있습니다. 그래서 먼저 문장을 작은 단위로 쪼개고(토큰화), 그 다음에는 숫자 벡터로 바꾸는 작업(임베딩)을 합니다. 이 과정에서 사용되는 기본 용어들을 하나씩 살펴보겠습니다.

- **토큰(Token)**: 자연어에서 처리의 최소 단위입니다. 보통은 단어 한 개, 또는 어

절 하나를 말합니다. 예를 들어 “나는 밥을 먹었다”라는 문장은 [“나는”, “밥을”, “먹었다”]로 쪼갤 수 있는데, 이때 각각의 텍스트 조각을 토큰이라고 합니다. 자연어 처리에서는 문장을 토큰 단위로 잘게 나누어 처리합니다.

- 문장(Sentence): 여러 개의 토큰이 모여 하나의 문장을 구성합니다. 예: [“나는”, “밥을”, “먹었다”] → 하나의 문장.
- 문서(Document): 여러 문장이 모이면 하나의 문서가 됩니다. 예: 블로그 글, 기사, 이메일 등
- 말뭉치(Corpus): 여러 문서가 모인 대량의 텍스트 데이터 세트입니다. 자연어 처리 모델을 학습시키기 위해서는 다양한 주제와 스타일 문서들을 대량으로 모은 말뭉치가 필요합니다.

~~어휘~~ ^{어휘사전} 집합(Vocabulary): 말뭉치에 등장한 중복되지 않은 토큰들의 모임입니다. 예를 들어 말뭉치에 ‘사랑’, ‘행복’, ‘기쁨’, ‘사랑’, ‘행복’이 있다면 어휘 집합은 [‘사랑’, ‘행복’, ‘기쁨’]으로 구성됩니다. 어휘 집합은 단어를 숫자로 바꿀 때 기준이 되는데, 보통 어휘 집합을 ‘Vocab’이라고 합니다.



[그림 9-1] 토큰, 문자, 문서, 말뭉치, 어휘 집합의 의미와 관계

토큰화와 임베딩

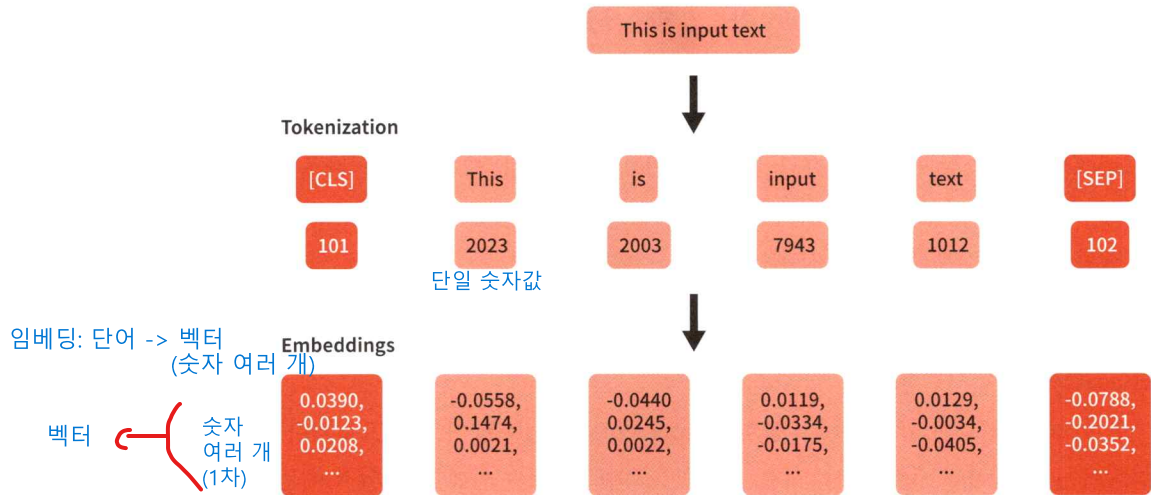
자연어 처리에서 모델을 만들기 전에 먼저 해야 할 작업이 바로 토큰화와 임베딩입니다.

- 토큰화(Tokenization): 문장을 작은 단위로 쪼개고 인덱스를 붙이는 과정입니다. 예를 들어 “This is input text”라는 문장을 단어 단위로 쪼개고, 쪼갠 단어를

단어 ---> 숫자
Token

머신러닝에서 vectorizer는 문장 단위였음.(CountVectorizer, TF-IDF)
 임베딩은 단어 단위가 벡터가 됨.

(단일)
 단어 -> 숫자 부여



[그림 9-2] 토큰화와 임베딩 과정 예시

특정 단어 -> (0,0,0) (축 3개 벡터)



각각 고유한 숫자로 바꿉니다. 예: This → 2023, input → 7943

- ★ 임베딩(Embedding): 숫자로 바꾼 단어에 의미를 담은 벡터를 입히는 과정입니다. 컴퓨터는 단순 숫자보다 의미가 있는 숫자 덩어리(벡터)를 더 잘 이해합니다. 예: 2023 → [0.0390, -0.0123, ...]

토큰화는 단어별로 나누고 단어가 '어떤 단어인지' 알려 주는 번호표이고, 임베딩은 '그 단어가 어떤 뜻인지' 알려 주는 숫자 벡터라고 생각하면 이해하기 쉽습니다.
 의미 부여

토큰화

단어

토큰화는 문장을 의미 단위로 쪼개는 작업이라고 설명했습니다. 예를 들어 “나는 밥을 먹었다”라는 문장을 [“나는”, “밥을”, “먹었다”]처럼 낱말 단위로 나눕니다. 이렇게 문장을 분리하면 컴퓨터가 텍스트를 더 잘 다룰 수 있습니다. 하지만 언어마다 그 방식이 조금씩 다릅니다.

영어는 띄어쓰기(공백) 기준으로 나누어도 대부분 토큰화가 가능합니다(예: “I am happy.” → [“I”, “am”, “happy”, “.”]). 영어는 단어가 명확히 구분되어 있어 split 함수만 사용해도 토큰화를 간단하게 수행할 수 있습니다. 마침표나 쉼표 같은 구두점(Punctuation)은 추가로 따로 처리하면 됩니다.

문장의 끝

영어에 대한 대표적인 토큰화 도구는 다음과 같습니다.

- **NLTK**(Natural Language Toolkit): 파이썬에서 가장 오래된 대표적인 자연어 처리 라이브러리로 영어 텍스트를 처리하는 데 필요한 다양한 기능을 제공합니다. 그중에서 `word_tokenize` 함수는 기본적인 단어 단위 토큰화 도구입니다.
- **spaCy**: 빠르고 정확한 최신 자연어 처리 라이브러리입니다. 속도가 매우 빠르며 토큰화뿐 아니라 품사 태깅(POS Tagging), 개체명 인식(NER) 등도 한꺼번에 수행할 수 있어 실무에서 많이 사용합니다.

두 라이브러리 모두 훌륭한 도구지만, 학습용으로는 NLTK, 프로젝트나 실무에서는 spaCy를 많이 사용합니다.

한국어는 토큰화와 함께 형태소 분석(Morphological Analysis)이 꼭 필요합니다. 한국어는 한 단어에 주어, 조사, 시제, 높임말 등 여러 요소가 함께 붙어 있기 때문입니다(예: “먹었습니다” → [“먹다”, “었”, “습니다”]).

한국어의 자연어 처리를 위해서는 형태소 분석기(예: **KoNLPy**, **Mecab**, **Okt** 등)를 사용해 단어를 쪼개고 품사 태깅까지 함께 수행해야 합니다. 한국어는 영어보다 토큰화가 복잡하기 때문에 형태소 분석기의 성능에 따라 결과가 많이 달라집니다.

한국어의 토큰화 도구로는 KoNLPy와 Kiwi 등이 있습니다.

- **KoNLPy**: 파이썬에서 한국어 형태소 분석기를 손쉽게 쓸 수 있게 해주는 라이브러리입니다. 여러 형태소 분석기(Okt, Mecab, Komoran, Hannanum 등)를 파이썬 코드로 쉽게 다룰 수 있게 도와줍니다.
- **Kiwi**(Korean Intelligent Word Identifier): 딥러닝 기반의 한국어 형태소 분석기입니다. 기존의 규칙 기반 분석기보다 더 유연하고 정확한 분석 결과를 제공하기 위해 만들어졌습니다. 특히 신조어, 띄어쓰기 오류, 오타자에 강합니다.

임베딩 문장

단어는 맥락 속에서 의미를 갖습니다. 언어학자 젤리그 해리스(Zellig Harris)는 1954년에 다음과 같은 말을 남겼습니다.

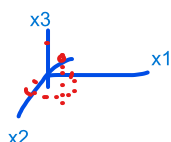
“비슷한 맥락(Context)에서 사용되는 단어들은 비슷한 의미(Meaning)를 가진다.”

예를 들어 ‘강아지’라는 단어가 자주 등장하는 문장에는 ‘귀엽다’, ‘산책’, ‘주인’ 같은 단어도 함께 등장합니다. 이런 식으로 같은 문맥에서 자주 나타나는 단어들은 의미

확률

적으로 유사하다는 관점을 분포 가설(Distributional Hypothesis)이라고 합니다. 이 가설은 오늘날 단어 임베딩의 핵심 철학이 되었습니다.

일반적으로 컴퓨터는 자연어 텍스트를 이해하지 못합니다. 사람은 문장을 읽으면 단어의 의미를 바로 파악하지만, 컴퓨터에게는 ‘강아지’나 ‘슬픔’ 같은 단어는 단순한 문자열에 불과합니다. 그래서 컴퓨터가 이 단어들을 이해할 수 있도록 숫자 벡터로 바꿔주는 과정, 즉 임베딩(Embedding)이 꼭 필요합니다.



단어 = (x_1, x_2, x_3)
= (1, 2, 3)

단어는 점 하나가 됨.
단어(점) 간 거리 측정 가능.



벡터(Vector)?

벡터는 여러 개의 숫자가 모여 하나의 의미를 표현하는 단위입니다. 예를 들어 어떤 사람의 정보를 [175cm, 70kg, 25세]처럼 하나의 숫자 묶음, 즉 하나의 단위로 나타낼 수 있습니다. 이것이 바로 벡터입니다. 자연어 처리에서도 단어(토큰)에 담긴 숫자를 하나로 묶어 하나의 벡터로 나타냅니다. 이 과정을 단어 임베딩이라고 합니다. 단어를 벡터로 바꾸면 컴퓨터가 단어의 위치, 유사도, 관계를 수학적으로 계산할 수 있습니다.

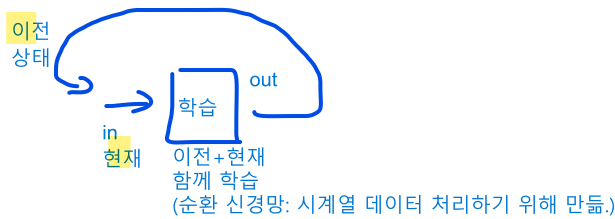
계산하여 오자 나오면 w를 줄일 수 있음.

토큰에 의미를 부여하는 임베딩 방식은 세 단계에 걸쳐 발전해 왔습니다.

- 등장 빈도 기반(BOW, TF-IDF): 토큰에 의미를 부여하는 가장 단순한 방법은 단어가 문서에 몇 번 등장했는지 세는 것입니다. 예를 들어 ‘사랑’이라는 단어가 문서 A에 5번, 문서 B에 1번 등장했다면 숫자 5, 1로 표현하는 방식입니다. 가장 초기에 시도된 방법이고 쉽게 적용할 수 있지만, 의미의 유사성, 단어의 순서나 문맥 등은 반영하지 못한다는 단점이 있습니다.
- 비슷한 문맥의 단어는 비슷한 의미(Word2Vec, GloVe): 주변 단어들(문맥)과 공통으로 등장하는 패턴을 학습해 벡터를 만듭니다. 예를 들어 ‘강아지’와 ‘고양이’는 모두 ‘귀엽다’, ‘집에서 키운다’와 함께 자주 등장하므로 의미적으로 가까운 벡터로 표현됩니다. 단점으로 문맥에 따라 의미가 달라지는 단어는 구분이 어렵습니다.
- 모델 학습 단계에 임베딩 포함: RNN 혹은 Transformer 모델에서 사용하는 방식입니다. 학습 과정에서 문장을 처음부터 끝까지 읽으며 단어의 문맥적 의미를 반영합니다. 최근에는 이 방식을 주로 사용합니다.

임베딩은 단어에 의미를 입히는 과정입니다. 단순한 등장 횟수부터 문맥을 반영하는 고급 임베딩에 이르기까지 꾸준히 발전해 왔습니다. 우리는 RNN을 이용한 모델링 과정에서 임베딩층이 포함된 언어 모델을 구성해 학습시킬 예정입니다.

순환 신경망



순환 신경망이란?

순환 신경망(RNN)은 이름 그대로 신경망 구조가 순환하는 딥러닝 모델입니다. 일
 딥러닝 안에 w 있음.
 반적인 신경망에서는 각각의 층이 입력을 한 번 받아 계산한 다음, 그 결과를 다음
 층으로 넘기고 끝납니다. 하지만 RNN은 시간적 순서(시점)가 있는 데이터를 처리
 시계열 데이터
 하도록 설계되어 있습니다. 특정 시점에 입력된 정보를 잠시 기억했다가 다음 시점
 으로 넘어갈 때 그 정보를 함께 사용합니다. 즉, 과거의 정보를 현재에 다시 반영하
 는 순환 구조를 가지고 있습니다.

책 읽는 상황을 떠올려 봅시다. 사람은 “하늘에 먹구름이 끼기 시작했다. 곧 비
 가 내리기 시작했다.”라는 두 문장을 읽을 때, 첫 번째 문장의 내용을 기억하고 있
 기 때문에 두 번째 문장이 자연스럽게 이해됩니다. ‘비가 내리기 시작했다’라는 말
 은 앞서 ‘먹구름이 끼었다’는 정보가 있기 때문에 의미가 이어집니다. 이처럼 사람
 은 문장을 읽을 때 앞의 내용을 기억한 채 다음 내용을 해석합니다. RNN도 마찬가
 지로 입력을 받을 때마다 이전 상태(State)를 기억하고 그 정보를 다음 시점의 입력
 처리에 활용합니다. 즉, 과거의 정보를 현재 계산에 반영하는 순환 구조를 통해 문
 맥을 이해합니다.

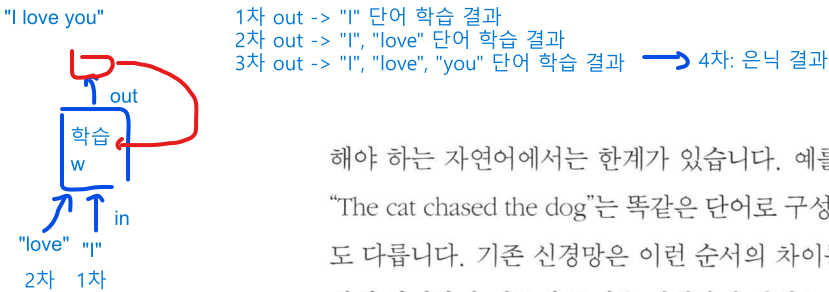
자연어를 처리함에 있어 단어 하나만 보고 문장에서 그 의미를 정확히 파악하기
 는 어렵습니다. 문장을 파악하고 이해하는 데 문맥(Context)과 순서(Sequence)가
 매우 중요하기 때문입니다. 예를 들어 다음 문장을 살펴볼까요?

“나는 오늘 아침 따뜻한 국을 먹었다.”

이 문장에서 ‘국’이라는 단어만 따로 보면 무슨 말인지 애매합니다. 그런데 ‘따뜻한’,
 ‘아침’, ‘먹었다’와 같은 단어와 함께 있다면 비로소 국이 음식이라는 사실을 이해합
 니다. 즉, 단어의 의미는 주변 단어와의 관계에서 결정됩니다. 또한 문장은 시간의
 흐름을 따라 하나씩 입력되기 때문에 컴퓨터는 이 단어들을 순서대로 이해할 수 있
 어야 합니다.

attention : 단어의 의미 파악.

그런데 기존 신경망(CNN, MLP)으로는 왜 부족할까요? 기존 신경망 구조는 입력
 을 한꺼번에 받아 한꺼번에 처리하는 방식입니다. 이 구조는 이미지나 숫자처럼 고
 정된 크기의 데이터를 처리할 때는 적합하지만, 단어의 순서나 문맥의 흐름을 고려



해야 하는 자연어에서는 한계가 있습니다. 예를 들어 "The dog chased the cat"과 "The cat chased the dog"는 똑같은 단어로 구성되어 있지만, 순서가 다르므로 의미도 다릅니다. 기존 신경망은 이런 순서의 차이를 인식하지 못하고 단어를 모두 동시에 처리하기 때문에 문맥을 이해하기 어렵습니다.

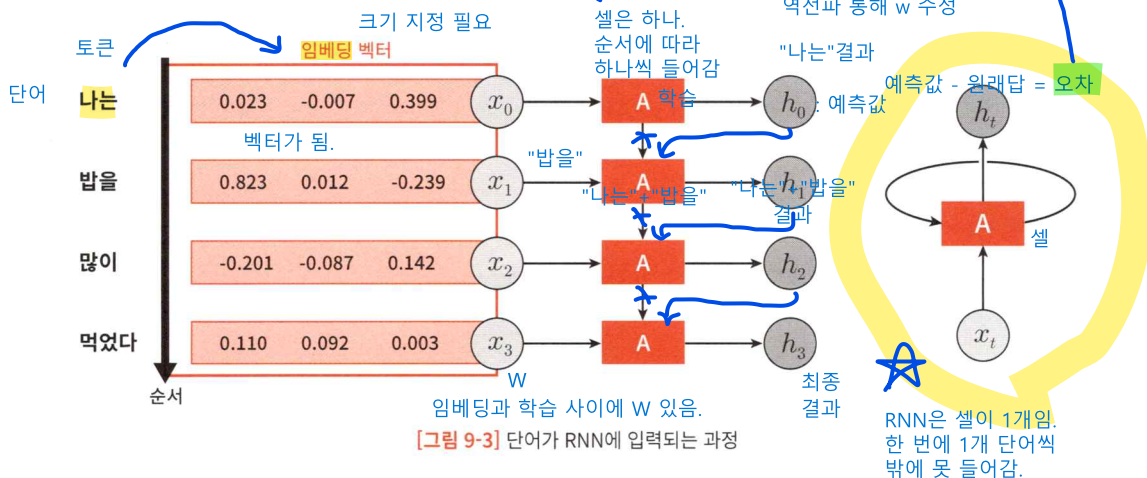
그래서 RNN이 필요합니다. RNN은 입력 문장에서 모든 단어를 한꺼번에 처리하지 않습니다. 순서대로 한 단어씩 읽고 이전 단어 정보를 저장하고 그 정보를 반영해 다음 단어를 처리하는 구조입니다. 즉, "The → dog → chased..."처럼 단어를 순서대로 입력받고 이전 내용을 기억하면서 문장을 점점 이해합니다.

이제 RNN의 기본 구조를 설명하겠습니다.

RNN의 기본 구조와 작동 원리

자연어는 단어의 순서에 따라 의미가 달라지기 때문에 이를 잘 처리하려면 **순차적인 구조를 이해하는 일이 중요합니다.**

[그림 9-3]을 보면 "나는 밥을 많이 먹었다"라는 문장의 단어들이 RNN에 순차적으로 입력되고 있습니다. 각각의 단어는 먼저 벡터로 변환(임베딩)되고 이 벡터들은 RNN에 순서대로 하나씩 입력됩니다. RNN에서는 입력된 단어 벡터를 기반으로 문맥을 파악합니다. 여기서 [나는, 밥을, 많이, 먹었다]가 순차적으로 입력된다는 의미에서 각각을 스텝(Step) 혹은 타임 스텝(Timestep)이라고 합니다.



[그림 9-3]을 보면서 각각의 단계에서 어떤 일이 일어나는지 자세히 설명하겠습니다.

임베딩

단어를 벡터로 바꾸고 RNN에 순서대로 넣는 과정

문장에서 **단어**는 먼저 **임베딩** 과정을 거쳐 **벡터로 변환**됩니다. 이 벡터는 단어의 의미를 수치로 표현한 것으로 딥러닝 모델이 텍스트를 이해할 수 있도록 돕는 중요한 과정입니다.

← 임의 값

“나는” → [0.023, -0.007, 0.399]

“밥을” → [0.823, 0.012, -0.239]

각각의 단어는 고정된 길이의 숫자 벡터로 표현할 수 있습니다. 숫자 벡터로 만들어진 임베딩 벡터들은 문장에서 순서를 유지한 채 하나씩 RNN에 입력됩니다.

예를 들어 첫 번째 단어 벡터 x_0 은 첫 번째 RNN 셀(A)에 입력되는데, 그 결과로 은닉 상태 h_0 이 계산됩니다. 다음으로 두 번째 단어 벡터 x_1 이 두 번째 셀에 들어가는데, 이때는 이전 단계의 은닉 상태 h_0 도 함께 입력으로 사용됩니다.

이 과정을 수식으로 표현하면 다음과 같습니다

$$h_t = \text{RNN}(x_t, h_{t-1})$$

학습

현재 결과

t: 현재

x_t : 현재 단어

h_{t-1} : 한 단계 이전 결과물

⇒ 순환 학습

즉, 각 시점 t 에서 현재의 단어 벡터 x_t 와 바로 이전 시점의 은닉 상태 h_{t-1} 을 입력으로 받아 새로운 은닉 상태 h_t 를 만듭니다. 이 h_t 는 문장의 의미를 점점 쌓아 가는 ‘중간 결과’라고 이해하면 됩니다.

RNN은 문장의 맥락을 앞에서부터 차례대로 학습하면서 나중에는 전체 문장의 의미를 요약한 은닉 상태를 얻습니다.

“나는” → h_0

“밥을(x_1)” → $h_1(x_1 + h_0)$ 을 기반으로 계산된 중간 결과물)

“많이(x_2)” → $h_2(x_2 + h_1)$ 을 기반으로 계산된 중간 결과물)

“먹었다(x_3)” → $h_3(x_3 + h_2)$ 을 기반으로 계산된 중간 결과물)

RNN은 이전 단계($t-1$ 시점)에서 계산된 은닉 상태 h_{t-1} 을 기억하고, 현재 단계(t 시점)의 입력인 x_t 와 함께 새로운 은닉 상태 h_t 를 만듭니다. **이전 정보를 다음 계산에 계속 전달하는 구조라는 의미**에서 바로 ‘**순환(Recurrent)**’이라는 이름이 등장합니다.

겉보기에는 여러 셀들이 순차적으로 이어져 있는 것처럼 보이지만, 실제로는 하나의 RNN 셀(또는 유닛)이 반복해서 호출되는 구조입니다. 즉, 같은 구조의 셀이

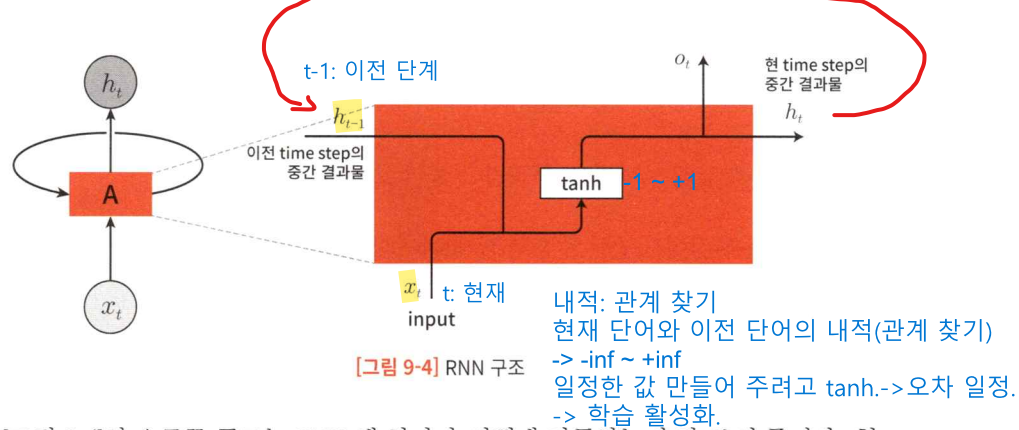


벡터와 w 내적

시그모이드 결점: 기울기 소실 효과(학습으로 기울기 줄어드는 게 아니라 미분 때문에 기울기 줄어듬.)
RNN에서 시그모이드 사용되면 긴 문장 학습이 어려움.(단어 100개면 역전파(미분) 100번.. 미분으로 기울기 0..)
=> 하이퍼볼릭 탄젠트 사용.(애도 미분에 약하긴 함. 애도 길게 할 수는 없음.)

RNN 자체가 긴 문장 불가. -> Transformer에서 해결책 나옴.

매 시점마다 다른 입력을 받아 내부 상태를 갱신하는 방식입니다. 이 과정에서 단어의 순서를 반영한 문맥 정보를 점차 쌓아 갈 수 있습니다.



[그림 9-4]의 오른쪽 구조는 RNN 셀 하나가 어떻게 작동하는지 잘 보여 줍니다. 현재 단어의 임베딩 벡터인 x_t 는 이번에 새로 들어온 입력이고, h_{t-1} 는 바로 직전 단어까지 계산해 얻은 중간 결과입니다. 이 두 정보를 RNN 셀에서 함께 처리해 새로운 결과인 h_t 를 계산합니다. 중요한 것은 이전 정보를 계속 기억하고 있다는 점입니다. 새로운 결과인 h_t 는 다음 셀로 넘어가면서 다음 단어와 함께 다시 계산에 사용됩니다. 즉, 이전 단어의 정보가 현재 단어에 반영되고 이 흐름이 문장 끝까지 이어지는 것이 바로 RNN의 핵심 구조입니다.

정리하면 RNN은 단어 하나하나를 순서대로 처리하면서 이전 상태 h_{t-1} 을 현재 입력 x_t 와 함께 입력해 새로운 상태 h_t 를 계산합니다. 이 과정을 통해 문장의 순서와 문맥을 자연스럽게 반영할 수 있습니다.

RNN 셀의 구조

앞에서 순환 신경망(RNN)이 순서가 있는 데이터를 처리하며 이전 상태 정보를 현재에 반영하는 구조라고 배웠습니다. 이번에는 RNN이 실제로 내부에서 어떻게 작동하는지, 그리고 수식이 어떤 의미를 가지는지 하나씩 살펴보겠습니다.

[그림 9-4]의 왼쪽 그림에 있는 RNN 셀 하나(A)는 다음 세 가지 요소로 입력과 출력을 처리합니다.

- 현재 들어온 단어의 벡터 x_t
- 이전까지의 상태를 담고 있는 h_{t-1} (이전 은닉 상태)
- 계산으로 얻은 새로운 h_t (현재 은닉 상태)

RNN 셀은 매 순간 현재 입력(x_t)과 과거의 기억(h_{t-1})을 함께 받아 새로운 기억(h_t)을 만드는 구조입니다. 이 구조가 바로 순환 구조의 핵심입니다.

은닉 상태값(Hidden State)을 계산하는 수식은 다음과 같습니다.

(은닉상태의 w) * 행렬곱 (이전 은닉상태) (현재데이터의 w) * 행렬곱 (현재 단어 벡터)

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

수식이 조금 복잡해 보이지만, 차근차근 풀어 보면 다음과 같습니다.

embedding_dim(Xt)에 따라 W_x 가 정해짐.



- h_{t-1} : 이전 시점에서 전달된 은닉 상태(과거의 정보)
- x_t : 현재 입력되는 단어의 임베딩 벡터($m \times 1$)
- h_t : 은닉층의 은닉 상태값($n \times 1$)
- o_t : 출력값($n \times 1$). 이후 설명
- W_h : 이전 은닉층에 곱한 가중치 행렬
- W_x : 입력에 곱한 가중치 행렬
- $\tanh$: 하이퍼볼릭 탄젠트(Hyperbolic Tangent) 함수는 입력값을 $-1 \sim 1$ 사이로 변환함. 모델에 비선형성을 부여하는 역할(활성화 함수 역할)

이 식으로 현재 입력된 단어와 이전까지의 흐름에 적절한 가중치를 곱해(학습된 비율을 반영해서) 새로운 정보(h_t)를 계산합니다.

RNN은 문장의 단어를 순서대로 하나씩 처리합니다. 이때 매 순간 새로운 h_t 를 만들고 그 h_t 는 다음 단어를 처리할 때 h_{t-1} 로 재사용됩니다. 이렇게 RNN은 이전 은닉 상태를 기억했다가 현재 은닉 상태에 반영하는 구조를 점점 누적하면서 문맥을 파악합니다.

수식에서 설명하지 않았던 o_t 는 h_t 와 같은 값입니다. 기호를 달리하는 이유는 다음 순서로 전달하는 의미(h_t)와 현 시점의 출력(o_t)을 구분하기 위함입니다. RNN은 시점(순서)마다 출력 o_t 를 생성할 수도 있고 마지막 시점의 은닉 상태 h_t 만으로 최종 예측을 할 수도 있습니다. 사용 목적에 따라 다르지만, 감정 분석이나 문장 분류 처럼 하나의 결과만 필요한 경우에는 보통 마지막 h_t 만 사용해 예측합니다.

$\tanh$ 는 -1 과 1 사이의 값을 출력하는 비선형 함수입니다. RNN에서 $\tanh$ 를 사용하는 이유는 계산 결과가 너무 커지지 않도록 막고 모델이 복잡한 패턴(비선형 패턴)을 학습하도록 도와주기 위함입니다. 또한 은닉 상태는 누적되기 때문에 너무 큰 값은 폭주(Exploding) 또는 소실(Vanishing) 위험이 있습니다. 따라서 $\tanh$ 는 이를 완화하는 역할도 합니다.

막지는 못함.

RNN 셀은 현재 단어와 이전 기억을 받아 새로운 기억(h_t)을 만듭니다. 이 구조는 단어의 순서를 반영하고 과거의 문맥을 현재에 반영하도록 설계한 것입니다. 수식은 단지 이 과정을 수학적으로 표현한 것이며 핵심은 이전 상태(h_{t-1})와 현재 입력(x_t)을 함께 활용한다는 점입니다.

RNN의 한계와 발전

RNN은 분명 순서가 있는 데이터를 처리하는 데 유용하지만, 실제로 학습을 시키면 몇 가지 어려움에 직면합니다.

미분해서 구하는 게 기울기. but 미분하면 기울기 0에 가까워짐. -> 오차 최소를 못 찾음 -> 학습x

- **기울기 소실 (Vanishing Gradient) 문제:** RNN은 단어를 순서대로 처리하면서 정보를 전달하지만, 너무 오래된 정보는 점점 희미해지고 사라지는 문제가 생깁니다. 예를 들어 “나는 어제 아침에 친구와 밥을 먹고 저녁에는 혼자 영화를 봤다”라는 문장이 있다고 합시다. 여기서 ‘어제 아침’이라는 정보는 맨 앞에 있는데, 문장이 길어지면 RNN은 이 정보를 끝까지 기억하지 못합니다. 기술적으로 역전파 (Backpropagation) 과정에서 초기 시점으로 갈수록 기울기(Gradient)가 점점 작아져 0에 수렴하면 결국 앞쪽 단어의 영향이 사라지는 현상이 발생합니다. tanh 함수로 이 문제를 조금 완화할 수는 있지만, 완벽한 해결책이 되지는 못합니다.

- **장기 의존성 문제 (Long-Term Dependency):** 기울기 소실 문제로 인해 **입력 문장이 길어질수록 앞쪽 정보가 뒤쪽 결과에 반영되지 못하는 현상**이 생깁니다. 이걸 장기 의존성 문제라고 합니다. 예를 들어 “민수는 [중략] 그래서 그는 매우 슬펐다”라는 문장이 있다고 합시다. 여기서 ‘그는’이 누구를 가리키는지 정확히 파악하려면 앞 문장에 있는 ‘민수’를 기억해야 합니다. 그런데 RNN은 이걸 잘 기억하지 못하고 엉뚱한 결과를 낼 수 있습니다.

- **병렬 처리의 어려움:** RNN은 단어를 **순서대로 하나씩 처리**하기 때문에 **학습 속도가 느리고 병렬 처리도 어렵습니다.** 기존 신경망(CNN 등)은 입력 여러 개를 한꺼번에 처리할 수 있지만, RNN은 ‘한 단어씩 순차적으로’ 처리해야 하므로 **GPU의 연산 효율이 떨어** 집니다.

RNN은 순차적인 데이터를 잘 처리할 수 있는 구조이지만, 기울기 소실과 장기 의존성 문제로 긴 문장을 기억하는 데 어려움이 있었습니다. 이로 인해 중요한 과거 정보를 끝까지 유지하기 어렵습니다. 이 문제를 해결하기 위해 등장한 구조가 바로

8단어 학습 도중
기울기 소실 문제 발생
(이전 기억(정보) 소실)
앞 단어의 의미 잊어버림.
-> 장기기억 불가.

기울기소실->미분->tanh 문제

병렬처리x -> GPU 사용 불가

Transformer

- 병렬처리 가능
- 순차처리 어렵?
- > 순서를 주입(positional encoding)하면
순서를 기억하며 병렬 처리 가능

LSTM(Long Short-Term Memory)과 GRU(Gated Recurrent Unit)입니다.

LSTM은 말 그대로 장기적인 기억(Long-Term Memory)을 유지하도록 설계된 구조입니다. LSTM의 핵심 아이디어는 '기억할지 말지, 잊을지 말지'를 결정하는 관문(Gate)을 두고 기억을 최대한 유지하는 일입니다. 이를 위한 장치(관문)는 다음 3가지입니다.

- 입력 게이트(Input Gate): 새로운 정보를 얼마나 기억할지 결정
- 망각 게이트(Forget Gate): 이전 정보를 얼마나 잊을지 결정
- 출력 게이트(Output Gate): 다음 단계에 어떤 정보를 전달할지 결정

단순히 h_t 만 전달하던 RNN과 달리, LSTM은 셀 상태(Cell State, C_t)라는 별도의 기억 공간으로 게이트를 정교하게 조절하면서 h_t 를 다음 단계로 넘깁니다.

GRU는 LSTM보다 구조가 조금 더 단순한 모델입니다. 게이트를 2개로 줄이고 셀 상태와 은닉 상태를 하나로 통합한 것이 특징입니다.

- 업데이트 게이트(Update Gate): 얼마나 기억할지 결정
- 리셋 게이트(Reset Gate): 이전 정보를 얼마나 반영할지 조절

LSTM은 복잡한 구조로 인해 연산하는 데 시간이 오래 걸립니다. GRU는 LSTM보다 가중치 수가 적으므로 계산은 빠른 반면, 성능은 LSTM과 비슷하다는 평가가 있어 실무에서는 빠르게 학습시킬 때 GRU를 많이 사용합니다.

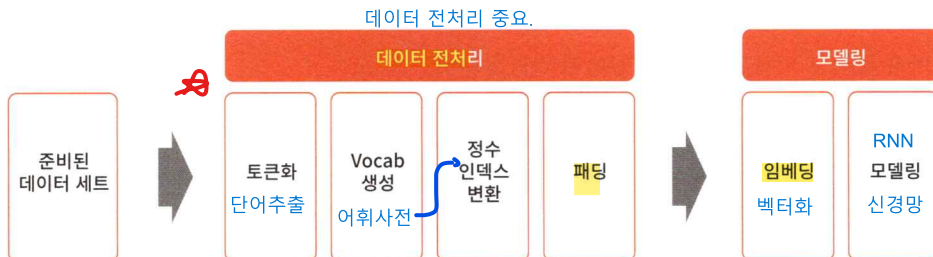
기울기 소실 및 장기 의존성 문제를 해결하기 위해서 LSTM과 GRU라는 개선된 순환 신경망 구조가 나왔다고 설명했습니다. 하지만 이들 역시 장기 의존성 문제를 완벽히 해결하지는 못합니다. 또한 RNN 계열의 알고리즘(RNN, LSTM, GRU)은 병렬 처리가 안 되는 문제가 있습니다. 이 문제점들은 2017년에 세상에 등장한 트랜스포머(Transformer)로 해결되었는데, 트랜스포머는 현재 chatGPT 같은 거대 언어 모델(Large Language Model, LLM)의 근간이 되었습니다.

언어 모델링 실습: 트위터 데이터 감정 분석 이진분류(금/부)

이번 절에서는 실제 데이터를 활용해 감정 분석 모델을 만듭니다. 먼저 언어 모델링의 전체 절차를 이해한 다음, 트위터 데이터를 대상으로 전처리부터 모델 구성까지 단계별로 살펴보겠습니다.

언어 모델링 절차

RNN을 활용한 언어 모델(RNN LM)을 만들려면 크게 두 단계가 필요합니다. 데이터 전처리, 모델 학습(임베딩 및 RNN 모델링)이 그것입니다. [그림 9-5]에서는 RNN 언어 모델의 절차를 다이어그램으로 나타냈습니다.



[그림 9-5] 언어 모델링 절차

각각의 단계에서 어떤 작업이 이루어지는지 구체적으로 알아보겠습니다.

데이터 전처리

텍스트 데이터로 모델을 학습시키려면 먼저 텍스트를 컴퓨터가 이해할 수 있는 형태로 바뀌어야 합니다. 텍스트 데이터의 전처리 과정은 다음 순서로 진행됩니다.

- **토큰화(Tokenization)**: 먼저 텍스트 문장을 단어 단위 또는 서브 워드 단위로 나눕니다. 이 작업으로 모델이 문장을 이해하기 쉽게 만듭니다.

예시: ["I love NLP", "Deep learning is fun", "AI is awesome"]
→ [{"I", "love", "NLP"}, [{"Deep", "learning", "is", "fun"}, [{"AI", "is", "awesome"}]]

0	...	N
단어	단어	단어

- **어휘 집합 생성(Vocab 생성)**: 토큰화된 전체 데이터를 살펴보고 **중복되지 않는 단어(토큰)**를 모아 하나의 사전을 만듭니다. 이 어휘 집합에서는 단어마다 고유 번호(ID)를 부여합니다.

단어:숫자. 단어가 key, 숫자가 value

예시: Vocab = {"I": 1, "love": 2, "NLP": 3, ..., "awesome": 9}

Transformer에서는 0~3은 다른 수. 4부터 시작됨.

순서가 있는 데이터

- **정수 인덱스 변환(Integer Encoding)**: Vocab을 이용해 문장의 각 단어를 숫자 시퀀스로 변환합니다. 이렇게 하면 단어가 아닌 숫자 리스트를 모델에 입력할 수 있습니다. 단어에 해당하는 인덱스 값 뽑기
예. "I" -> 1

[x,x,x] 이래야 벡터

예시: ["I", "love", "NLP"] → [1, 2, 3]
["Deep", "learning", "is", "fun"] → [4, 5, 6, 7]



시퀀스 길이 맞추기(Padding): 문장의 길이가 모두 같지 않기 때문에 짧은 문장에는 0이나 PAD 토큰을 덧붙여 길이를 맞춥니다. 문장의 길이를 맞춰야 미니배치 단위로 학습할 때 행렬 연산이 가능합니다.

예시: [1, 2, 3] → [1, 2, 3, 0]
[4, 5, 6, 7] → [4, 5, 6, 7]

학습은 1개 셀의 반복. 문장 단어 수만큼. 문장에 따라 반복해야 할 수 다름.(for문으로 처리 불가.)
max_length로 제일 긴 문장 기준으로 처리.
짧은 문장은 시퀀스 길이 맞추기.

모델링

모델링 단계에서는 다음 두 가지 작업이 이루어집니다.

정수인덱스

- **임베딩(Embedding)**: 각각의 단어를 숫자로 바꾸었다고 해도 그 자체로는 의미를 알 수 없습니다. 그래서 각각의 정수 인덱스를 의미가 있는 벡터(임베딩 벡터)로 변환하는 작업을 합니다. 벡터로 변환하면 컴퓨터는 단어 간의 의미 관계를 더 잘 이해할 수 있습니다.
- **모델 학습(RNN 모델 구성)**: 마지막으로 임베딩된 시퀀스를 RNN에 입력해 학습을 진행합니다. 앞에서 살펴보았듯이 RNN은 단어의 순서를 기억하면서 다음 단어를 예측하거나 문장의 감정 같은 특징들을 분류하는 작업을 수행합니다.

벡터는 학습시킬 수 있음.

seq2seq sequence to sequence

첫번 같은..

RNN + Linear + 이진
순서 유지 위해 RNN 사용.

행렬 변환 통해 단어와 w 내적.
w는 오차가 최소가 되게 함.
단어가 의미 있는 쪽으로 이동.

이와 같이 모델링 단계에서는 텍스트 데이터를 딥러닝 모델이 이해할 수 있는 형태로 바꾸고 RNN으로 언어 모델을 생성합니다.

다음 과정에서는 언어 모델링 과정을 실습과 함께 살펴봅니다.

환경 준비 및 데이터 불러오기

코랩에서 새 코랩 파일을 생성하고 파일 이름은 "9일차_언어모델링실습.ipynb"로 합니다. 라이브러리 불러오기 → 디바이스 준비 → 사용자 정의 함수 생성 → 데이터 로딩순으로 진행합니다. 그리고 이번 실습에서는 런타임을 GPU로 설정하고 진행하겠습니다.

코랩 상단 메뉴에 있는 [런타임] - [런타임 유형 변경]을 클릭하고 나온 [런타임 유형 변경] 대화상자에서 'T4(GPU)'를 선택합니다.

라이브러리 불러오기

먼저 라이브러리를 불러옵니다. 자연어 처리를 위해 새롭게 추가한 라이브러리와 함수들이 있는데, 하나씩 살펴보겠습니다

CODE

```
# 기본 라이브러리
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import *

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# 자연어 처리를 위해 추가된 부분
import nltk ①
from nltk.tokenize import word_tokenize ②
from nltk.corpus import stopwords ③
from tqdm.auto import tqdm ④
from collections import Counter ⑤
import string ⑥
nltk.download("punkt") ⑦
nltk.download("punkt_tab") ⑧
nltk.download("stopwords") ⑨
np.set_printoptions(linewidth=200) ⑩
```

OUTPUT

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
```

- ① nltk: Natural Language Toolkit으로 파이썬에서 자연어 처리를 쉽게 해주는 라이브러리입니다.
- ② word_tokenize: 문장을 단어 단위로 잘라 주는 함수입니다. 예: "I love AI" → ["I", "love", "AI"]
- ③ stopwords: 'the', 'is', 'and'처럼 의미가 적고 분석에 필요하지 않은 단어들을 모아 둔 목록입니다.
- ④ tqdm.auto: 반복문의 진행 상황을 시각적으로 보여 주는 라이브러리입니다.

- ⑤ Counter: 단어의 등장 빈도를 손쉽게 계산해 주는 파이썬 내장 클래스입니다.
- ⑥ string: 이 라이브러리는 문장 부호(., !, ? 등)를 다룰 때 유용하게 사용하는 문자열 관련 상수와 함수를 제공합니다.
- ⑦ punkt: 토큰화에 필요한 사전 학습된 모델입니다.
- ⑧ punkt_tab: 일부 환경에서 추가로 필요한 토큰나이저 설정입니다.
- ⑨ stopwords: 영어 등에서 불용어 리스트를 가져오기 위한 리소스입니다
- ⑩ 넘파이 배열을 출력할 때 한 줄에 표시하는 문자 수를 200자로 늘립니다. 이 옵션을 사용하는 이유는 이후 넘파이 배열을 보기 좋게 화면에 출력하기 위함입니다.

언어 모델링을 위한 기본 환경 설정을 위해 텍스트 전처리와 데이터 분석에 필요한 주요 라이브러리들을 폭넓게 불러왔습니다. 특히 nltk 관련 리소스를 사전에 다운로드함으로써 이후 토큰화와 불용어 제거 작업을 원활하게 수행할 수 있도록 준비합니다. 이 단계는 텍스트 데이터를 효과적으로 다루기 위한 필수적인 준비 과정입니다. 디바이스 설정은 이전과 동일하므로 여러분이 직접 코드를 작성해 보기 바랍니다.

사용자 정의 함수: make_DataSet 수정

이번에는 사용자 정의 함수(make_DataSet, train, evaluate, dl_learnig_curve) 생성 및 디바이스 설정입니다. make_DataSet 함수를 제외한 나머지 함수와 디바이스 설정은 이전과 동일하므로 5일 차 186~187쪽을 참조해 입력합니다. make_DataSet 함수는 다음과 같이 변경합니다.

```
CODE
def make_DataSet(x_train, x_val, y_train, y_val, batch_size = 32) :
    # 데이터 텐서로 변환
    x_train_tensor = torch.tensor(x_train, dtype=torch.long)
    y_train_tensor = torch.tensor(y_train, dtype=torch.long)
    x_val_tensor = torch.tensor(x_val, dtype=torch.long)
    y_val_tensor = torch.tensor(y_val, dtype=torch.long)

    # TensorDataset 생성: 텐서 데이터셋으로 합치기
    train_dataset = TensorDataset(x_train_tensor, y_train_tensor)

    # DataLoader 생성
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle = True)
    return train_loader, x_val_tensor, y_val_tensor

| ① x의 데이터 타입을 정수형(long)으로 지정합니다.
```

왜 입력 데이터(x)의 데이터 타입을 정수형으로 지정할까요? 이유는 입력 정보가 바로 Vocab의 인덱스이기 때문입니다. 모델에서 정수형 인덱스는 임베딩층을 거치면서 벡터값(실수형)으로 변환됩니다. 이 변환은 다음에 나오는 데이터 전처리와 모델 선언 과정에서 다시 한번 설명하겠습니다.

데이터 가져오기

이번 실습에서는 트위터에 올라온 문장을 분석해 그 안에 담긴 감정을 6가지 가운데 하나로 분류하는 모델을 만듭니다. 다음과 같이 데이터를 불러옵니다.

CODE

```
data = pd.read_csv('https://bit.ly/emotions_csv')
data.head()
```

OUTPUT

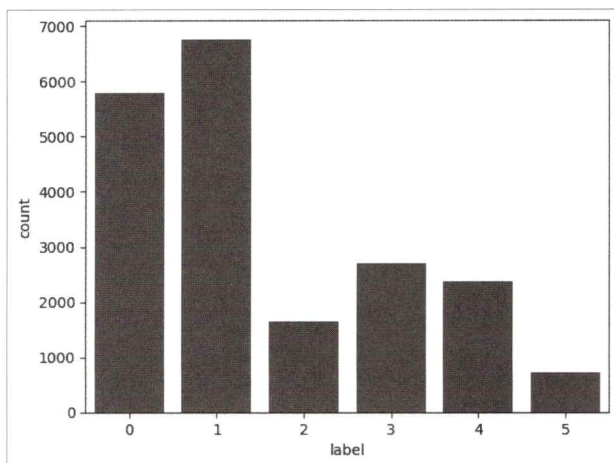
text	label
i didnt feel humiliated	0
i can go from feeling so hopeless to so damned hopeful just from being around someone who cares and is awake	0
im grabbing a minute to post i feel greedy wrong	3
i am ever feeling nostalgic about the fireplace i will know that it is still on the property	2
i am feeling grouchy	3

코드를 실행하면 상위 5개의 트위터 데이터를 확인할 수 있습니다. 총 20,000개의 트윗 문장으로 이루어진 이 데이터는 각각의 트윗 문장에 대한 6가지 종류의 감정 상태를 표현한 목푯값(Label)을 가지고 있습니다. 목푯값은 정수로 인코딩된 값이며, 각각은 0(Sadness), 1(Joy), 2(Love), 3(Anger), 4(Fear), 5(Surprise)를 의미합니다.

트위터 데이터 세트에서 감정 상태별로 데이터가 몇 건씩 있는지 그래프로 확인하겠습니다.

CODE

```
sns.countplot(x = 'label', data = data) ①
plt.show()
```



① countplot 메서드는 값의 빈도수(건수)를 막대 그래프로 그려 줍니다('label' 열의 값 0~5의 빈도수)

그래프를 보면 **감정 데이터는 균등하지 않고** 0(sadness)과 1(joy)이 상대적으로 많습니다. 반면 2(love)와 5(surprise)는 상대적으로 적습니다. 목פות값의 분포가 균등하지 않고 불균형을 이루고 있는데, 이런 모습을 범주 불균형(Class Imbalance)이라고 합니다. 보통 딥러닝에서는 불균형 문제를 해소하도록 전처리한 후에 모델링을 수행합니다. 하지만 이번 차시에서는 언어 모델링에 초점을 두므로 범주 불균형 문제를 따로 다루지는 않겠습니다.

이어서 언어 모델링을 수행하기 전에 트윗 문장들의 전처리 과정을 진행합니다.

데이터 전처리 1

RNN의 데이터 전처리 과정은 앞에서 진행했던 전처리 과정과 약간 차이가 있습니다. 특히 데이터 전처리 1은 자연어 모델만의 독특한 처리 과정을 포함합니다. 트위터 데이터의 토큰화, 어휘 집합 생성, 정수 인덱스 변환, 패딩으로 이어지는 전처리 과정을 차근차근 코드로 구현합니다.

토큰화

모델이 문장을 처리하려면 단어 단위로 쪼개는 과정, 즉 토큰화(Tokenization)가 필요합니다. 이 단계에서는 단순한 텍스트를 단어 목록(리스트) 형태로 변환해 다

음 단계에서 사용할 수 있게 합니다. 토큰화와 관련된 전처리 과정은 다음과 같은 단계들로 이루어집니다.

1. 텍스트 전체를 소문자로 변환
2. 문장을 단어 단위로 나누기(tokenize)
3. 분석에 필요 없는 단어들(불용어, 구두점) 제거
4. 최종 토큰 리스트를 새로운 열(tokens)로 저장

이 4가지 과정의 전처리 코드를 다음과 같이 작성합니다.

CODE

```
# 불필요한 단어(Stopwords & 구두점) 집합 준비
stop_words = set(stopwords.words("english")) ①
punctuations = set(string.punctuation) ②
set : 중복제거할 때 사용.

# 전처리 함수
def preprocess_text(text):
    tokens = word_tokenize(text.lower()) ③
    tokens = [word for word in tokens if word not in stop_words and word not in
               punctuations] ④
    return tokens

# 데이터에 전처리 함수 적용, Series - 하나씩 꺼내서
data["tokens"] = data["text"].apply(preprocess_text) ⑤
data.head()
```

OUTPUT

	text	label	tokens
0	i didnt feel humiliated	0	[didnt, feel, humiliated]
1	i can go from feeling so hopeless to so damned...	0	[go, feeling, hopeless, damned, hopeful, aroun...
2	im grabbing a minute to post i feel greedy wrong	3	[im, grabbing, minute, post, feel, greedy, wrong]
3	i am ever feeling nostalgic about the fireplac...	2	[ever, feeling, nostalgic, fireplace, know, st...
4	i am feeling grouchy	3	[feeling, grouchy]

- ① 영어에서는 자주 등장하지만, 의미 분석에는 도움이 되지 않는 단어들만 모은 집합을 만듭니다. 예: ["is", "the", "an", "and", "a", ...]
- ② 마침표, 쉼표, 물음표 등 구두점 집합을 만듭니다. 예: [",", ":", "?", "!", ...]
- ③ 모든 글자를 소문자로 변환합니다(대소문자의 차이를 없애기 위함).
- ④ 불용어(stop_words)와 구두점(punctuations)을 제외한 단어만 남기는 명령입니다.
- ⑤ 트위터 문장마다 앞에서 정의한 전처리 함수 preprocess_text를 적용해 토큰 리스트를 생성한 다음, 새로운 열(tokens)에 저장합니다.

코드를 실행하면 기존의 text 열값이 tokens 열값으로 토큰화됩니다. 즉, 각각의 문장은 단어 단위로 나뉘고 불필요한 데이터는 제거되어 리스트 형태의 데이터만 남습니다.

tokens 기반

Vocab 만들기

이제 토큰화된 단어들을 숫자로 바꾸는 작업을 진행합니다. 이때 필요한 도구가 바로 어휘 집합(Vocabulary, Vocab)입니다. Vocab 만들기는 전체 데이터에 등장하는 고유한 단어들을 모으고 각각의 단어에 고유한 번호(인덱스)를 부여하는 작업입니다. Vocab으로 토큰 리스트를 숫자 시퀀스로 바꿀 수 있습니다.

등장 빈도순으로 정렬된 어휘 집합을 만들고 숫자 시퀀스로 변환하는 코드를 다음과 같이 작성합니다.

```
CODE
[ ] 단어 [ ]
[전체 단어] all_words = [word for tokens in data["tokens"] for word in tokens] ①
word_counts = Counter(all_words) ②
vocab = {word: idx + 1 for idx, (word, _) in enumerate(word_counts.most_common())} ③
vocab["<PAD>"] = 0 ④
vocab["<UNK>"] = len(vocab) ⑤
print(f"어휘 집합 크기: {len(vocab)}")
```

OUTPUT

어휘 집합 크기: 16947

- ① data["tokens"]는 각각의 문장을 토큰화한 단어 리스트입니다. for 문으로 문장의 단어 모두를 하나로 모아 all_words라는 리스트를 만듭니다.
- ② Counter 함수는 리스트에서 각각의 단어가 몇 번 등장하는지 세는 도구입니다. Counter 함수를 실행하면 {단어: 등장 횟수}와 같이 파이썬의 딕셔너리 형식으로 변환된 데이터가 변수 word_counts에 저장됩니다. 예: {"love": 1, "movie": 2, "weather": 1, "great": 1, "today": 1}
- ③ most_common 메서드는 단어를 등장 횟수가 많은 순서대로 정렬한 리스트를 반환합니다. 그리고 enumerate 함수로 각각의 단어에 순서를 매기고 고유한 숫자(ID)를 부여합니다. 여기서 +1을 한 이유는 나중에 패딩(PAD)을 0번 인덱스로 사용하기 위해서입니다. 따라서 실제 단어에 부여되는 숫자는 1번부터 시작하게 됩니다.
- ④ 보통 패딩은 인덱스 0으로 합니다.
- ⑤ 학습할 때는 없었던 새 단어가 나올 경우를 대비해 <UNK> 토큰을 만듭니다.

Vocab의 생성 절차는 문장의 모든 단어를 하나로 모으고 각각의 단어가 얼마나 자주 나왔는지 빈도수를 계산합니다. 그리고 자주 등장하는 단어부터 순서대로 정수 인덱스를 부여합니다. 마지막으로 특수 토큰을 추가합니다. 예를 들어 패딩을 위한

토큰 <PAD>에는 0번 인덱스를 부여하고, 사전에 없는 단어가 나오면 <UNK>(알 수 없는 단어)를 부여합니다. 이렇게 생성한 Vocab의 단어 수는 모두 16,947개입니다.

정수 인덱스로 변환

모델은 단어를 이해하지 못하며 숫자로 변환해야 학습할 수 있습니다. 그래서 문장을 구성하는 단어들을 정수 인덱스(숫자)로 변환하는 과정이 필요합니다. 이때 사용하는 도구가 앞에서 만든 어휘 집합(Vocab)입니다. 각각의 단어에는 Vocab을 만들 때 부여한 고유한 숫자(인덱스)가 있습니다. 문장의 단어들을 하나씩 Vocab에서 찾아 해당 숫자로 바꾸면 됩니다. 만약 단어가 Vocab에 없다면 <UNK> 인덱스로 대체됩니다.

CODE

```
def text_to_sequence(tokens):
    return [vocab.get(word, vocab["<UNK>"]) for word in tokens] ①
data["sequences"] = data["tokens"].apply(text_to_sequence) ②
data[["text", "tokens", "sequences"]].head()
```

OUTPUT

	text	tokens	sequences
0	i didnt feel humiliated	[didnt, feel, humiliated]	[50, 1, 510]
1	i can go from feeling so hopeless to so damned...	[go, feeling, hopeless, damned, hopeful, aroun...	[31, 2, 408, 2987, 431, 44, 54, 1565, 1252]
2	im grabbing a minute to post i feel greedy wrong	[im, grabbing, minute, post, feel, greedy, wrong]	[4, 2988, 1100, 183, 1, 381, 308]
3	i am ever feeling nostalgic about the firelac...	[ever, feeling, nostalgic, fireplace, know, st...	[78, 2, 561, 4628, 6, 13, 3363]
4	i am feeling grouchy	[feeling, grouchy]	[2, 918]

- ① text_to_sequence는 tokens(문장을 토큰화한 단어 리스트) 열의 단어 리스트를 숫자 리스트로 변환하는 함수입니다.
- ② data["tokens"] 열의 모든 리스트에 text_to_sequence 함수를 적용합니다. 결과는 sequences라는 새로운 열에 저장됩니다.

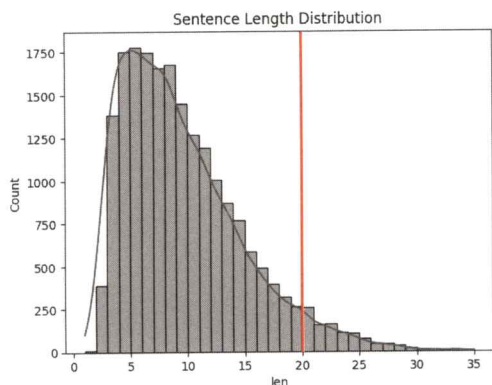
코드의 실행 결과로 나온 데이터의 일부를 보면 text가 토큰화되고 다시 정수로 인코딩되었음을 알 수 있습니다.

패딩(문장 길이 맞추기)

RNN은 입력 데이터의 길이가 모두 동일해야 학습할 수 있습니다. 하지만 실제 텍스트 데이터는 문장마다 단어 수(길이)가 제각각입니다. 예를 들어 문장 A가 7개 단어, 문장 B가 4개 단어, 문장 C가 12개 단어라면 길이가 다르기 때문에 하나의 배치(Batch)로 묶어 학습하기 곤란합니다. 그래서 모든 문장의 길이를 일정한 길이로

맞춰주는 작업, 즉 패딩(Padding)을 수행해야 합니다. 패딩은 문장이 짧다면 뒤에 0(PAD 토큰)을 붙여 정해진 길이에 맞추는 작업입니다. 또한 문장이 길다면 필요한 만큼 잘라 길이를 맞춘 다음 배치로 묶어 학습시킵니다.

학습을 위해 문장을 동일하게 맞추려면 어디까지 잘라야 할지 또는 얼마나 채워야 할지에 대한 기준이 필요합니다. 그래서 문장의 길이를 정하는 두 가지 방법을 소개합니다.



Text 1	101	14	66	53	7	87	34	14	37	31	17	9	21	102
Text 2	101	91	20	15	17	98	36	81	23	0	· 긴 부분은 자르고 · 짧은 부분은 채우고			
Text 3	101	87	34	14	37	0	0	0	0	0				
Text 4	101	23	74	2	67	102	54	243	82	163				

[그림 9-6] 문장의 길이 분포를 감안하기

- 토큰 길이 분포 고려: 문장 길이 분포(히스토그램)를 확인하고 전체의 95% 정도를 커버하는 길이를 선택합니다(예: 대다수 문장이 20단어 이내라면 길이를 20으로 설정)
- 시스템 제약 고려: 메모리 용량이나 모델 학습 속도를 고려해 문장의 길이, 즉 토큰 수를 보통 50~512개 사이로 설정합니다. 너무 길면 학습 시간이 오래 걸리고, 너무 짧으면 중요한 정보가 잘릴 수 있으므로 적절한 길이를 선택하는 것이 필요합니다.

이번 단계에서는 첫 번째 패딩(Padding) 방법을 적용합니다. 먼저 분포를 그리고 95%에 해당하는 지점을 찾아보겠습니다.

다음은 토큰의 길이를 계산해 열로 추가하는 코드입니다.

CODE

```
data["len"] = data["tokens"].apply(len) ①
data.head()
```

OUTPUT

	text	label	tokens	sequences	len
0	i didnt feel humiliated	0	[didnt, feel, humiliated]	[50, 1, 510]	3
1	i can go from feeling so hopeless to so damned...	0	[go, feeling, hopeless, damned, hopeful, aroun...	[31, 2, 408, 2987, 431, 44, 54, 1565, 1252]	9
2	im grabbing a minute to post i feel greedy wrong	3	[im, grabbing, minute, post, feel, greedy, wrong]	[4, 2988, 1100, 183, 1, 381, 308]	7
3	i am ever feeling nostalgic about the fireplac...	2	[ever, feeling, nostalgic, fireplace, know, st...	[78, 2, 561, 4628, 6, 13, 3363]	7
4	i am feeling grouchy	3	[feeling, grouchy]	[2, 918]	2

① 토큰의 길이(개수)를 세어 열로 추가합니다.

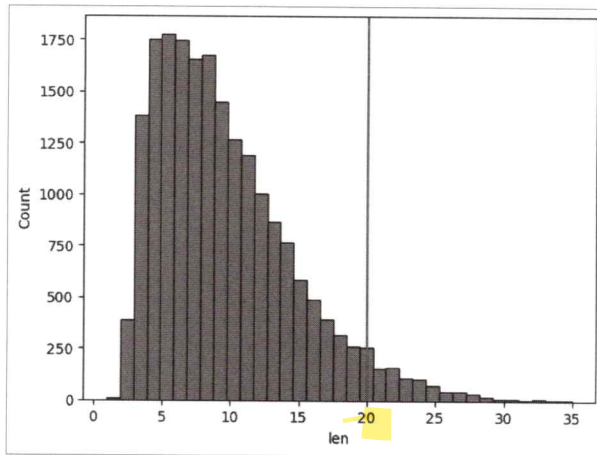
토큰의 길이에서 95% 지점을 찾고 히스토그램 위에 표시합니다.

CODE

```
p95 = np.percentile(data['len'], 95) ①
print('95% 지점', p95)
sns.histplot(x = 'len', data = data, bins = 35) ②
plt.axvline(p95, color = 'r') ③
plt.show()
```

OUTPUT

95% 지점 20.0



- ① percentile 메서드로 data['len']의 95% 지점을 찾아 p95에 저장합니다.
- ② histplot 메서드를 이용해 data['len']에 대한 히스토그램을 그립니다.
- ③ ②의 히스토그램에서 95%에 해당하는 값(여기서는 20) 위치에 수직선(plt.axvline)을 추가합니다.

그래프를 보면 95% 지점의 토큰 수(len)가 20임을 한눈에 알 수 있습니다.

이어서 20자리로 자르고 0으로 채우는 패딩을 수행합니다. 여기서는 파이토치의 패딩 함수 대신 케라스(Keras)의 pad_sequences 함수를 사용합니다. 이유는 파이

Tensorflow

토크의 패딩 함수는 사용법이 약간 복잡하고 별도의 텐서 조작이 필요한 반면, 케라스 함수는 한 줄로 간단하게 패딩을 적용할 수 있습니다(참고로 케라스(Keras)는 파이토치와 함께 딥러닝 모델링을 지원하는 대표적인 라이브러리입니다).

CODE

```
from keras.utils import pad_sequences ①
max_len = 20 ②
sequences = pad_sequences(data["sequences"],
                           maxlen=max_len, ③
                           padding='post', ④
                           truncating='post', ⑤
                           value=vocab["<PAD>"]) ⑥
```

- ① 케라스의 pad_sequences 함수를 불러옵니다.
- ② 최대 시퀀스 길이를 설정합니다.
- ③ 모든 문장을 20단어의 길이로 맞춥니다.
- ④ 문장 끝(post)에 0을 채워 패딩합니다(예: [2, 5] → [2, 5, 0, 0, ...]).
- ⑤ 길이가 20을 초과하는 단어는 잘라냅니다.
- ⑥ 패딩할 때 숫자를 vocab["<PAD>"]로 지정합니다. 여기서는 0입니다.

패딩 코드를 실행하고 패딩의 결과를 다음과 같이 출력합니다.

CODE

```
print(sequences[:10])
```

OUTPUT

```
[[ 50  1 510  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [ 31  2 408 2987 431 44 54 1565 1252  0  0  0  0  0  0  0  0  0]
 [  4 2988 1100 183  1 381 308  0  0  0  0  0  0  0  0  0  0  0]
 [ 78  2 561 4628  6 13 3363  0  0  0  0  0  0  0  0  0  0  0]
 [  2 918  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
 [ 16  2  8 454 207 218 61  0  0  0  0  0  0  0  0  0  0  0]
 [ 16 233 8278 98 8279 748 16 2698 1394 64 1725 38 1 3 298  0  0  0  0]
 [  1 342 18 1814 696 91 168 257  0  0  0  0  0  0  0  0  0  0]
 [5826 122 1 5826 5827 49 46 893 4629  0  0  0  0  0  0  0  0  0]
 [  1 540  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]]
```

코드를 실행하면 상위 10개 문장의 데이터 길이는 모두 20입니다. 데이터의 길이가 20보다 작은 문장은 나머지가 0으로 채워져 있습니다.

지금까지 언어 모델링을 위한 특별한 전처리 작업인 토큰화(Tokenize), Vocab 만들기, 정수 인덱스로 변환, Padding(문장 길이 맞추기) 작업을 단계별로 수행했습니다.

데이터 전처리 2

다음은 종전의 모델링 작업과 유사한 전처리 과정입니다. 데이터 세트의 준비가 완료되었으니 x와 y 준비, 학습과 검증 데이터 세트 분할, 데이터 로더 및 검증 텐서 데이터를 다음과 같이 준비합니다.

CODE

```
x = np.array(sequences) ①
y = np.array(data["label"]) ②
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size = 0.2)
train_loader, x_val_ts, y_val_ts = make_DataSet(x_train, x_val, y_train, y_val, 32)
```

① sequences는 각각의 문장을 토큰화하고 정수 시퀀스로 변환하고 패딩까지 적용한 결과물입니다. 이를 넘파이 배열로 변환해 x에 담습니다.

② y는 data["label"]을 가져와 역시 넘파이 배열로 변환합니다.

전처리 과정을 모두 마쳤습니다. 이제 모델 구조를 설계하고 학습시키겠습니다.

모델 구조 설계

지금부터 모델 구조를 설계하고 학습, 검증 및 평가로 이어지는 과정을 진행합니다. 앞에서 배운 모델링과 다른 점은 바로 모델 구조입니다.

LSTM LM 모델의 전체 구조

이번에는 감정 분류 모델의 전체 구조를 살펴봅니다. 앞에서 배운 단어 임베딩과 RNN 구조가 실제 코드로는 어떻게 구현되는지 직접 확인하겠습니다.

우선 다음과 같이 모델 클래스를 작성합니다.

CODE

```
class SimpleLSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_size, output_size):
        super(SimpleLSTMClassifier, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim) ①
        self.lstm = nn.LSTM(embedding_dim, hidden_size, batch_first=True) ②
        self.fc = nn.Linear(hidden_size, output_size) ③
    def forward(self, x):
        x = self.embedding(x) ④
        _, (hidden, _) = self.lstm(x) ⑤
        out = self.fc(hidden[-1]) ⑥
        return out
```

이번에 구현할 감정 분류 모델은 임베딩층→LSTM층→완전 연결층(출력층)의 구조로 이루어져 있습니다. 구조는 간단하지만, 처음 등장하는 `nn.Embedding`(임베딩층)이나 `nn.LSTM`(LSTM층) 같은 함수는 조금 어렵게 보일 수 있습니다.

각각의 코드를 설명하기 전에 감정 분류 모델 구조 각 층의 입력 특징 수와 출력 특징 수를 표로 정리했습니다.

[표 9-1] 모델 구조 예시에서 각 층의 입출력 특징 수

층	입력 특징 수	출력 특징 수
<code>nn.Embedding</code>	<code>vocab_size</code>	<code>embedding_dim</code>
<code>nn.LSTM</code>	<code>embedding_dim</code>	<code>hidden_size</code>
<code>nn.Linear</code>	<code>hidden_size</code>	<code>output_size</code>

각각의 층이 서로 연결되어 있으므로 임베딩층의 출력 특징 수는 다음에 연결할 LSTM층의 입력 특징 수가 되고, LSTM층의 출력 특징 수는 다시 완전 연결층의 입력 특징 수가 됩니다.

이 부분을 기억하면서 각각의 층을 차근차근 살펴보겠습니다.

① `nn.Embedding`(임베딩층)

임베딩은 모델이 텍스트를 이해할 수 있도록 단어를 수치화된 벡터로 바꾸는 과정이라고 설명했습니다. 문장을 숫자 시퀀스로 바꾸는 것은 전처리 과정(정수 인코딩 및 패딩)에서 이미 진행했습니다. 모델의 임베딩층에서는 단어의 의미를 벡터로 표현합니다.

CODE

```
self.embedding = nn.Embedding(vocab_size, embedding_dim)
```

임베딩층의 `vocab_size`는 전체 어휘 집합의 크기입니다. 예를 들어 어휘 집합에 5,000개의 고유 단어가 있다면 이 값은 5,000입니다. `embedding_dim`은 각각의 단어를 표현할 벡터의 크기입니다. 보통 100, 200과 같이 설정하며 차원이 높을수록 단어 간 의미를 더 정밀하게 표현할 수 있습니다.

임베딩층은 모델 구조에 포함되어 학습 과정에서 가중치를 업데이트합니다. 이 층은 텍스트 데이터를 벡터 형태로 변환해 딥러닝 모델이 문장의 의미를 효과적으

로 파악하도록 도와줍니다. 이 과정에서 의미가 비슷한 단어들은 서로 유사한 벡터로 표현됩니다.

② nn.LSTM(LSTM층)

임베딩으로 벡터로 바뀐 단어들을 순서대로 처리하는 층이 바로 LSTM층의 역할입니다. LSTM층은 문장을 구성하는 단어들을 순서대로 입력으로 받으며 그 순서와 문맥을 이해하려고 합니다.

LSTM은 앞에서 본 정보를 기억하면서 다음 단어를 해석하도록 도와줍니다. 특히 RNN의 단점인 기울기 소실 문제를 완화하기 때문에 긴 문장에서도 앞뒤 문맥을 비교적 잘 파악합니다.

CODE

```
self.lstm = nn.LSTM(embedding_dim, hidden_size)
```

embedding_dim은 임베딩 벡터의 차원으로 입력의 크기를 의미합니다. hidden_size는 LSTM이 문맥 정보를 얼마나 잘 기억할지 결정하는 은닉 상태(Hidden State)의 크기입니다. 이 값이 크면 클수록 LSTM이 더 많은 정보를 담을 수 있지만, 계산량도 그만큼 늘어납니다.

LSTM층은 단어의 순서와 문맥을 이해하는 데 있어 핵심적인 역할을 합니다. 특히 긴 문장에서 앞뒤 단어 간의 연결을 잘 파악하도록 도와줍니다.

③ nn.Linear(완전 연결층)

LSTM에서 얻은 마지막 은닉 상태(Hidden State)는 문장의 전체 정보를 요약한 결과라고 볼 수 있습니다. 이 은닉 상태값을 바탕으로 완전 연결층(출력층)에서 최종 결괏값을 예측합니다.

CODE

```
self.fc = nn.Linear(hidden_size, output_size)
```

hidden_size는 LSTM에서 출력된 은닉 상태의 크기입니다. 문장의 의미가 여기에 압축되어 담겨 있습니다. output_size는 분류하려는 결괏값의 개수입니다. 예를 들어 감정 분류에서는 6개의 감정 범주가 있으므로 output_size=6으로 설정됩니다.

정리하면 완전 연결층은 LSTM층에서 나온 문장 요약 정보(은닉 상태)를 바탕으

로 최종 예측을 수행합니다. 이 층에서 모델은 감정 범주별로 점수를 계산하고 가장 높은 점수를 가진 범주를 예측 결과로 선택합니다.

④ forward 메서드의 임베딩 단계

이 단계에서는 입력으로 들어온 문장 데이터를 임베딩층을 통해 의미 있는 벡터로 변환합니다.

```
CODE
x = self.embedding(x)
```

입력 `x`는 `[batch_size, seq_len]` 형태이며 이 값들은 단어를 숫자로 바꾼 정수 인덱스입니다. 이 정수 인덱스 각각은 `nn.Embedding`을 통과하면서 고정된 크기의 의미 벡터로 변환됩니다. 변환 결과는 `[batch_size, seq_len, embedding_dim]` 형태의 3차원 텐서가 됩니다.

예를 들어 입력 문장이 `[2, 5, 10]`이라는 정수 시퀀스라면 임베딩 후에는 다음과 같이 각각의 벡터로 바뀝니다.

예: 입력 문장 → `[2, 5, 10]`, 임베딩 후 → `[[0.12, -0.03, ...], [0.21, 0.07, ...], [0.01, 0.02, ...]]`

임베딩 벡터는 단어의 의미를 숫자 벡터로 표현해 LSTM이 문장의 의미를 더 잘 이해하도록 도와줍니다.

⑤ forward 메서드의 LSTM 단계

이번 단계에서는 임베딩된 문장을 LSTM에 입력하고 그 결과로 문장의 의미를 요약한 벡터를 얻습니다. LSTM은 문장을 구성하는 단어들을 순서대로 하나씩 처리하며 각 시점마다 은닉 상태와 내부 상태를 계산합니다.

```
CODE
_, (hidden, _) = self.lstm(x)
```

이 코드는 LSTM의 출력값 중 필요한 값만 선택해 사용하는 방식입니다. LSTM은 총 3개의 출력값을 반환합니다. 이 코드에서는 그중 두 번째 값만 `hidden` 변수에 담고 나머지 두 값은 사용하지 않기 때문에 `_`로 처리했습니다.

- 첫 번째 출력: 각 시점의 은닉 상태를 모두 담은 텐서(문장의 모든 단어에 대한 정보)
- 두 번째 출력: 마지막 단어까지 모두 처리한 후의 최종 은닉 상태(문장 전체의 요약 정보)
- 세 번째 출력: LSTM의 내부 셀 상태(기억 장치 상태)

이 예제에서는 문장 전체를 대표하는 하나의 벡터, 즉 최종 은닉 상태(Hidden)만 사용합니다. 감정 분류처럼 문장 단위로 예측하는 작업에서는 단어별 정보가 아니라, 문장 전체의 의미를 담고 있는 요약 벡터 하나만 있으면 충분하기 때문입니다.

⑥ forward 메서드의 출력층

이제 마지막 단계입니다. LSTM을 통과해 얻은 문장 요약 벡터를 출력층(완전 연결층)에 전달해 최종 감정 분류 결과를 얻습니다.

CODE

```
out = self.fc(hidden[-1])
```

hidden[-1]은 LSTM의 마지막 층에서 나온 최종 은닉 상태 벡터입니다. 이 벡터는 문장의 의미를 잘 요약한 정보라고 볼 수 있습니다. LSTM은 여러 층으로 구성될 수 있기 때문에 hidden 텐서는 [num_layers, batch_size, hidden_size] 형태입니다. 이 중에서 가장 마지막 층(-1)의 은닉 상태를 꺼내 사용하면 됩니다.

출력층에서는 이 벡터를 입력으로 받아 최종적으로 분류할 감정 범주 수만큼의 값을 출력합니다. 이 결과는 감정 범주 각각의 확률 또는 점수로 사용됩니다.

다음 [표 9-2]는 지금까지의 모델링 과정과 그 과정에서 다루어지는 데이터의 구조(shape)를 요약한 정보입니다. 모델이 어떻게 데이터를 처리하는지 한눈에 이해하는 데 도움이 됩니다.

[표 9-2] 모델의 구조와 흐름, 출력 크기

단계	설명	출력 크기 예시
임베딩	단어 인덱스를 의미 벡터로 변환	[32, 20, 100]
LSTM	시퀀스를 따라 정보 처리	hidden[-1] → [32, 128]
FC Layer(출력층)	은닉 상태를 감정 범주로 분류 변환	[32, 6]
최종 출력	문장별 감정 예측값	[32, 6]

학습 및 모델 평가

모델 선언

지금까지 만든 모델 클래스를 사용해 모델을 선언합니다. 기존의 모델 선언 코드와 달리 LSTM 모델은 vocab_size와 embedding_dim을 추가로 입력값으로 지정할 수 있습니다.

```
CODE
vocab_size = len(vocab) ①
embedding_dim = 100 ②
hidden_size = 128 ③
output_size = 6 ④
learning_rate = 0.001
model = SimpleLSTMClassifier(vocab_size, embedding_dim, hidden_size,
                             output_size).to(device) ⑤
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
```

- ① 어휘 집합 크기
- ② 단어 임베딩 차원
- ③ LSTM 은닉층 크기
- ④ 감정 범주의 개수(anger, fear, joy, love, sadness, surprise)
- ⑤ LSTM 모델 선언

모델을 준비했으니 이제 모델 학습 및 평가 단계로 넘어갑니다.

학습 및 성능 평가

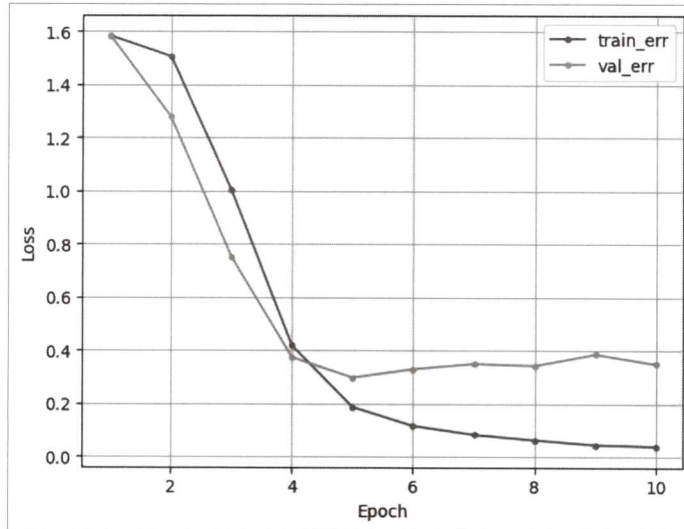
모델을 선언했으니 이제 모델을 반복 학습시키며 성능을 확인합니다. LSTM 모델 역시 기본적인 학습 구조는 5일 차에서 살펴본 다중 분류 모델과 같습니다. 다음과 같이 코드를 작성합니다.

```
CODE
epochs = 10
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model, loss_fn, optimizer, device)
    val_loss, _ = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)
    tr_loss_list.append(tr_loss)
    val_loss_list.append(val_loss)
    print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")

dl_learning_curve(tr_loss_list, val_loss_list)
```

OUTPUT

```
Epoch 1, train loss : 1.582555, val loss : 1.582607
Epoch 2, train loss : 1.504480, val loss : 1.277368
Epoch 3, train loss : 1.003537, val loss : 0.751714
...
Epoch 8, train loss : 0.062404, val loss : 0.342364
Epoch 9, train loss : 0.044441, val loss : 0.385510
Epoch 10, train loss : 0.038292, val loss : 0.348621
```



학습 곡선을 보면 에포크 4까지 급격히 검증 오차(val_err)가 줄어들다가 이후 오차가 살짝 증가하면서 학습이 종료됩니다.

이어서 예측 및 평가를 위한 코드도 작성합니다.

CODE

```
# 예측
_, pred = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)
print(type(pred))
pred = pred.softmax(dim = 1) ①
pred = np.argmax(pred.cpu().numpy(), axis = 1) ②
```

OUTPUT

```
<class 'torch.Tensor'>
```

- ① 예측값으로 저장된 pred는 텐서 데이터입니다. 따라서 텐서를 지원하는 softmax 메서드를 사용했습니다. nn.functional.softmax(pred, dim=1)과 동일합니다.
- ② pred.cpu().numpy(): pred는 gpu에 있으므로 cpu로 옮기고 넘파이 배열로 변환합니다. argmax 메서드로 가장 확률이 큰 인덱스를 pred에 저장합니다.

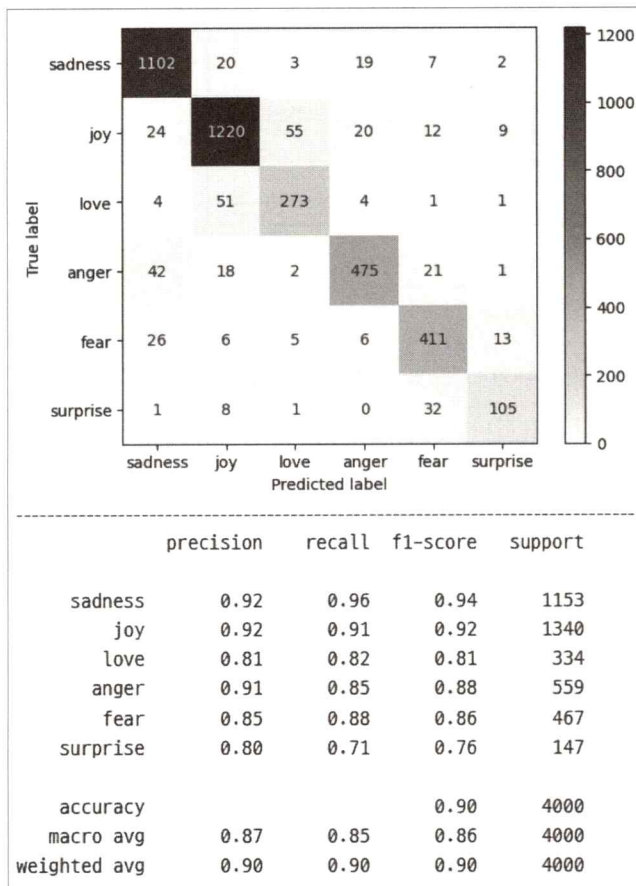
감정 분류 모델을 평가하기 위해 혼동 행렬(Confusion Matrix)과 분류 결과 보고서(Classification Report)의 결과도 출력합니다. 다음과 같이 작성합니다.

CODE

```
# confusion matrix 시각화
label_list = ['sadness', 'joy', 'love', 'anger', 'fear', 'surprise'] ①
cm = confusion_matrix(y_val_ts.cpu().numpy(), pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=label_list)
disp.plot(cmap="Blues")
plt.show()
print('-'*100) ②

# classification_report
print(classification_report(y_val_ts.cpu().numpy(), pred, target_names=label_list))
```

OUTPUT



① 6개의 감정을 리스트에 담습니다. 혼동 행렬과 분류 결과 보고서 함수의 입력으로 사용합니다.

② 혼동 행렬 결과와 분류 결과 보고서 결과 사이에 경계선을 긁습니다.

모델의 전체 정분류율은 약 90%로 트윗 감정 분류에서 전반적으로 안정적인 성능을 보입니다. 특히 슬픔(sadness)과 기쁨(joy)은 정밀도와 재현율이 모두 높아 긍정, 부정 반응 분석에 효과적으로 활용할 수 있습니다. 분노(anger)와 사랑(love)도 비교적 잘 구분되어 고객 불만 탐지나 긍정 반응 분석에 도움이 됩니다.

반면 두려움(fear)과 놀람(surprise)은 다른 감정에 비해 성능이 낮아 실제 위기 상황에서 사용자의 불안 신호나 긴급 반응을 제대로 감지하지 못할 가능성이 있습니다. 따라서 이 모델은 고객 만족도 분석이나 마케팅 반응 모니터링에는 충분히 활용할 수 있지만, 위기 대응과 같은 민감한 영역에서는 보완 학습이 필요합니다.

정리

이번 차시에서는 자연어 처리의 기초 개념부터 순환 신경망(RNN)의 구조와 언어 모델링에 이르기까지 딥러닝의 자연어 처리 분야를 빠르게 배웠습니다.

먼저 자연어 처리란 무엇인지 그리고 문장을 처리하기 위해 텍스트를 어떻게 정제하고 숫자 형태로 바꾸는지 알아보았습니다. 이어서 RNN이 문장의 순서를 어떻게 기억하고 문맥을 반영하는지 그 작동 원리를 알아보았습니다. 또한 RNN의 한계를 보완한 LSTM, GRU 같은 개선된 구조도 함께 살펴보았습니다. 마지막으로 실제 트윗 데이터를 활용해 6가지 감정을 분류하는 언어 모델링 실습을 통해 자연어 처리 모델이 실제로 어떻게 만들어지고 작동하는지 직접 체험했습니다.

자연어는 복잡하고 섬세한 정보를 담고 있는 데이터입니다. 이번 차시의 내용만으로 자연어 처리를 모두 익힐 수는 없습니다. 하지만 텍스트를 수치화하고 문맥을 이해하고 의미를 예측하는 인공지능의 기본 흐름 정도는 이해할 수 있었기를 바랍니다.

다음은 이 책의 마지막 차시로서 비지도학습의 기본 구조인 오토인코더(Auto encoder)를 살펴보겠습니다.